AUDITORIA ESTÁTICA

Relatório de Auditoria de Segurança —

Horium — sistema de gestão de horário escolar

15 de outubro de 2026
Escopo: código e configuração versionados neste repositório
Método: revisão estática com cadeia de evidência. Sem alteração de aplicação.

1. Escopo, stack e método

A auditoria examinou o repositório Horium como uma aplicação de página única. Não há servidor HTTP próprio: o navegador fala direto com o Supabase usando a chave anônima pública. O controle de acesso efetivo está em `schema.sql` (RLS, triggers e funções) e nos serviços do cliente que chamam essas APIs.

Stack detectada

Camada	O que o repositório usa
Linguagem	TypeScript e SQL (PostgreSQL)
Frontend	React 19 e Vite 6. Sem Next.js e sem backend Node.
Runtime	Navegador. Build estático publicado com vercel.json.
Dados	Supabase: Auth, PostgREST, Storage e RPC.
ORM	Não há. Consultas via supabase-js e SQL no schema.
Autenticação	supabase.auth (senha, OTP, sessão persistida).
Autorização	RLS por auth.uid(), is_admin() e checagens dentro das RPC.
Isolamento	user_id nas policies. Não há tenant, workspace nem RLS por organização.
Arquivos	Buckets privados receipts e tickets-attachments.
Externos	Supabase, fontes Google, avatares Dicebear, payload PIX no cliente.
Docker / K8s / IaC / CI	Ausentes neste repositório.
Secrets	.env.example com placeholders. Chave real só via VITE_* no build.
Testes	Vitest em services/__tests__ e utils/__tests__.

Fronteiras de confiança

- Navegador para o bundle Vite: todo segredo com prefixo VITE_ fica público.
- Bundle para a API Supabase: a chave anon só abre o que a RLS e as RPC permitirem.
- API para o Postgres: policies, triggers e funções SECURITY DEFINER.
- API para o Storage: policies em storage.objects por pasta do usuário.
- Não há workers de servidor nem pipeline de CI versionado.

Nota metodológica

Um achado só entra como confirmado quando a cadeia está no código: entrada controlável, fluxo até um recurso sensível e ausência ou falha do controle que deveria barrar essa entrada. Policies, triggers, RPC e o mascaramento no React foram lidos em conjunto. Padrão perigoso isolado, tabela sem escritor e chave anon de papel público não foram contados.

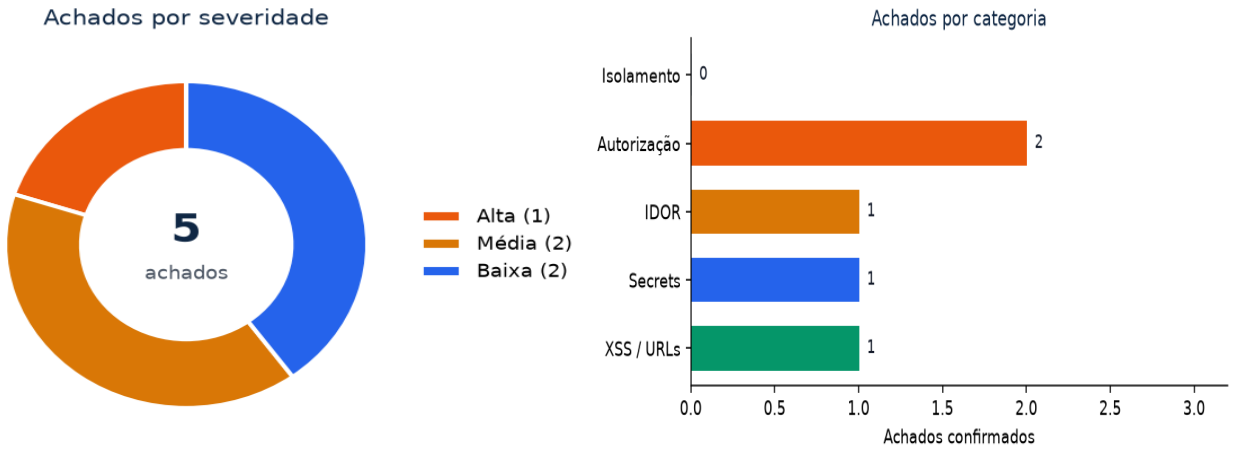
Limitações

- O banco em produção não foi consultado. O veredito vale para schema.sql como definição versionada.
- Não houve exploit ao vivo, varredura de dependências nem teste de penetração.
- O bundle de produção não está no repositório. O código-fonte do cliente foi tratado como o que o navegador recebe.
- O histórico Git contém apenas .env.example além do código. O conteúdo desse arquivo foi conferido nos commits a5db3e7 e 1962d12.

2. Resumo executivo

Foram confirmados 5 achados: 1 alto, 2 médios e 2 baixos. Nenhum é crítico e nenhum permite, com o schema atual, ler a grade de outro usuário. O risco principal é o paywall da solução horária: um usuário autenticado grava a própria grade com `is_licensed = true` e a solução permanece em `schedules.data`, porque o trigger que isola as aulas roda antes do trigger que zera a licença.

Severidade	Quantidade	Leitura
CRÍTICA	0	Nenhum acesso anônimo ao dado de outro usuário foi demonstrado.
ALTA	1	Paywall da solução contornável por qualquer conta autenticada.
MÉDIA	2	Solução permanece legível após revogação; URL de comprovante sem allowlist.
BAIXA	2	Caminho de comprovante sem pasta do dono; PIX com fallback fixo.



Distribuição por categoria: Autorização 2, IDOR 1, Secrets 1, XSS/URLs 1, Isolamento cross-tenant 0. A rosca omite a fatia crítica porque o valor é zero; o total do centro é a soma dos achados confirmados.

3. Pontos fortes

Os controles abaixo foram lidos no fluxo correspondente e seguram o ataque que o comentário do schema descreve.

Controle	Evidência	O que impede
RLS de schedules	schema.sql:83-100	SELECT, UPDATE e DELETE de grade alheia. A policy exige auth.uid() = user_id ou admin.
RLS de licenses	schema.sql:210-229	Usuário comum só lê a própria licença. INSERT direto foi removido.
Preço no servidor	schema.sql:588-677	request_license_order ignora valor enviado pelo cliente e confere o dono da grade.
RPC administrativas	schema.sql:401-403, 439-441, 504-506, 546-548, 574-576, 826-828	get_admin_licenses, approve, delete, tickets e send_admin_notification chamam is_admin().
schedule_solutions	schema.sql:138-143	A tabela da solução só tem policy de admin. O cliente não lê essa tabela direto.
Storage	schema.sql:763-801	Upload e leitura exigem a primeira pasta igual a auth.uid(), ou admin na leitura.
Conteúdo de notificação	schema.sql:254-272	Usuário comum não altera title, message nem user_id.

Controle	Evidência	O que impede
Tickets	schema.sql:300-310	INSERT exige o próprio user_id. UPDATE de status é só admin, também na RPC.
XSS de esquema	adminLicenseService.ts:34-47 e AdminPanelPage.tsx:143	javascript:, data: e afins são recusados antes de href, img e iframe.
Texto de UI	Layout.tsx:311-317	Título e mensagem de notificação entram como texto React, sem innerHTML.
Gate de admin na UI	Layout.tsx:71 e App.tsx:508	O botão usa a RPC is_current_user_admin. Abrir a view ADMIN sem ser admin não passa nas RPC.

4. Pontos fracos

- O paywall depende da ordem alfabética de dois triggers BEFORE. O isolamento vê is_licensed ainda verdadeiro e deixa fixedLessons em schedules.data.
- A policy de SELECT da grade não olha a licença. Rejeitar ou expirar o pagamento não apaga a solução já gravada.
- O fallback do cliente, em scheduleService.getScheduleSolution, devolve data.fixedLessons se a RPC negar.
- Comprovante e anexo aceitam URL HTTPS arbitrária com token=, e o painel abre esse endereço.
- A chave PIX de recebimento tem fallback fixo quando VITE_PIX_KEY não existe no build.

5. Cobertura

Categoria	Escopo analisado	Controles encontrados	Itens	Achados	Cobertura
Isolamento	7 tabelas, policies, triggers, 10 RPC, 2 buckets	RLS por user_id e is_admin(); pasta do Storage	Tabelas, RPC e jobs	0	Integral
Autorização	Gates isAdmin, isLicensed e activeLicenseStatus cruzados com RPC e RLS	is_admin() nas RPC; trigger de is_licensed; máscara no React	Admin, paywall, aprovação e tickets	2	Integral
IDOR	Todos os acessos .from, .rpc e .storage nos services e páginas	Filtro user_id no cliente e RLS no banco	27 operações e 10 RPC	1	Integral
Secrets	Fonte, .env.example, vercel.json e histórico Git	Placeholders no arquivo atual; sem service_role	Árvore atual e .env.example	1	Integral
XSS / URLs	innerHTML, href, src, iframe e HTML de e-mail	React escapa texto; getSafeHttpsUrl bloqueia esquemas	UI, avisos, comprovantes e anexos	1	Integral

Não aplicável como superfície separada: Docker, docker-compose, Helm, Kubernetes, Terraform e CI/CD. Motivo: não existem nesses formatos no repositório. O deploy descrito é o header de vercel.json sobre o build estático.

Analisado e não contabilizado

- public.profiles possui SELECT para authenticated com USING (true) em schema.sql:28. Não há INSERT, trigger nem escrita dessa tabela no repositório, então não há cadeia até dado de outro usuário.

- O commit a5db3e7 gravou em .env.example a URL do projeto Supabase e um JWT cujo papel é anon. O arquivo atual voltou a placeholders no commit 1962d12. A chave anon é a mesma classe de credencial pública do bundle e não eleva privilégio. O valor não é reproduzido aqui. Convém rotacioná-la se aquele projeto ainda estiver ativo.
- App.tsx:508 renderiza a área admin sem um if isAdmin. As RPC e a RLS recusam o usuário comum. Esconder o botão não é o controle, e o controle de servidor existe.
- Content-Security-Policy em vercel.json usa script-src 'unsafe-inline'. Não há sink HTML controlado pelo usuário que torne isso explorável.

6. Matriz de ownership

Não há handlers HTTP locais. Cada linha é uma operação que o navegador pode chamar com a chave anon e o JWT do usuário. “OK” significa que a RLS ou a RPC barra o objeto de outro usuário antes da operação sensível.

Operação	Objeto	Ownership
scheduleService. getSchedules	schedules + licenses	OK — filtro user_id e RLS
scheduleService. getScheduleSolution	RPC e fallback em schedules.data	RPC OK; fallback devolve aulas se ainda estiverem em data
scheduleService. saveSchedule	schedules por id	Dono OK; paywall falha na ordem dos triggers
scheduleService. delete, restore, purge e trash	schedules por id	OK — filtro user_id e RLS
adminLicenseService. getAllLicenses	get_admin_licenses	OK — is_admin()
adminLicenseService. approveLicense	approve_license_rpc	OK — is_admin()
adminLicenseService. rejectLicense	UPDATE licenses por id	OK — RLS só admin
adminLicenseService. deleteLicense	delete_license_rpc	OK — is_admin()
adminLicenseService. getPendingCount	COUNT licenses	OK — RLS limita ao próprio usuário
adminLicenseService. getReceiptSignedUrl	storage receipts	Dono da pasta OK; URL https com token= passa direto
ticketService. createTicket	tickets + storage	user_id OK; image_url não é validada no banco
ticketService. getAllTickets	get_admin_tickets ou SELECT	OK — RPC de admin; fallback limitado pela RLS
ticketService. getUserTickets	tickets	OK — filtro user_id e RLS
ticketService. updateTicketStatus	RPC ou UPDATE	OK — is_admin() e policy de admin
ticketService. getAttachmentSignedUrl	storage ou URL pronta	URL http(s) com token= é devolvida sem allowlist
notificationService. get, count e mark	notifications	OK — RLS e trigger de conteúdo
notificationService. createNotification	send_admin_notification	OK — is_admin()

Operação	Objeto	Ownership
auditLogService. getMyAuditLogs	audit_logs	OK — filtro user_id e RLS
PlansPage. handleUploadReceipt	storage + request_license_order	Preço e dono da grade OK; caminho do arquivo não exige a pasta do uid
LicenseSelection e App.loadLicenseStatus	licenses	OK — .eq(user_id) e RLS
Storage receipts e tickets	storage.objects	OK para arquivo na pasta do uid

7. Achados detalhados

Severidade	Arquivo:linha	Descrição
ALTA	schema.sql:166 e schema.sql:175	Trigger de isolamento roda antes do trigger que zera is_licensed e grava a solução sem licença.
MÉDIA	schema.sql:99 e adminLicenseService.ts:166	SELECT da grade não exige licença vigente. Rejeitar o pagamento não apaga fixedLessons.
MÉDIA	schema.sql:647 e ticketService.ts:88	Comprovante e anexo aceitam URL HTTPS arbitrária aberta pelo administrador.
BAIXA	schema.sql:647	O caminho do comprovante não precisa estar na pasta auth.uid().
BAIXA	pages/PlansPage.tsx:41	Sem VITE_PIX_KEY, o PIX de cobrança usa um e-mail fixo.

1. [ALTA] Paywall contornável pela ordem dos triggers

Categoria. Autorização. O dado é do próprio usuário; o que falta é a licença.

Arquivo:linha. schema.sql:166 (decisão de manter as aulas) e schema.sql:175 (trigger criado depois, porém com nome que roda antes).

Fluxo. Usuário autenticado faz INSERT ou UPDATE em public.schedules com is_licensed verdadeiro e data.fixedLessons preenchido. No PostgreSQL, triggers BEFORE do mesmo evento disparam em ordem alfabética do nome. trg_isolate_schedule_solution executa antes de trg_protect_schedule_license. O isolamento vê a licença ainda verdadeira, copia as aulas e não remove a chave do JSON. O segundo trigger então força is_licensed para falso, tarde demais.

Por que é vulnerável. O flag que deveria ser sempre falso para não admin ainda é verdadeiro no momento em que a solução é preservada. A policy de SELECT devolve data ao dono. A chave anon do bundle torna essa chamada direta possível, fora da máscara do React.

Quem explora. Qualquer conta autenticada, inclusive cadastro aberto por authService.signUp.

Pré-condições. schema.sql aplicado como está, com os dois nomes de trigger atuais. Não exige ser admin nem ter pagamento aprovado.

Impacto. A solução horária, que o schema trata como conteúdo pago, fica armazenada e legível em schedules.data sem licença. O retorno do próprio INSERT já pode trazer o JSON.

Controles considerados. protect_schedule_license_status realmente zera o flag. A RLS impede ler a grade de outro usuário. schedule_solutions não é legível pelo usuário. getSchedules no React zera fixedLessons quando isLicensed é falso. Nenhum desses apaga o JSON já gravado nem impede o SELECT direto.

Correção. Remover fixedLessons de schedules.data sempre, para não admin, sem consultar is_licensed. Ou nomear e ordenar o trigger de proteção para rodar primeiro. Não devolver a solução por SELECT da tabela; manter só a RPC.

Trecho.

```
IF NOT COALESCE(NEW.is_licensed, false) THEN
NEW.data := NEW.data - 'fixedLessons';
END IF;
-- schema.sql:166
CREATE TRIGGER trg_isolate_schedule_solution -- roda antes
CREATE TRIGGER trg_protect_schedule_license -- roda depois
```

O cliente amplia o efeito em services/scheduleService.ts:122-132: se a RPC get_schedule_solution falha, o código lê schedules.data.fixedLessons e devolve o array. A RPC em si, nas linhas 700-719, recusa quem não tem licença.

2. [MÉDIA] Solução continua legível depois da revogação

Categoria. Autorização.

Arquivo:linha. schema.sql:99-100 e services/adminLicenseService.ts:164-167.

Fluxo. Uma licença já aprovada deixa is_licensed verdadeiro. Nesse estado o trigger de isolamento mantém fixedLessons dentro de data. rejectLicense só executa UPDATE em licenses (status Rejeitado, valid_until nulo). Nada atualiza schedules.is_licensed nem remove a chave JSON. A policy de SELECT continua true para o dono. get_schedule_solution até zera is_licensed, mas só se alguém chamar essa RPC.

Por que é vulnerável. Expirar ou rejeitar o pagamento não é o caminho de leitura. Quem já teve a licença ligada continua podendo selecionar a coluna data.

Quem explora. O próprio titular da grade, autenticado.

Pré-condições. A flag is_licensed chegou a ser verdadeira e um save ocorreu nesse estado. Não serve para quem nunca foi licenciado se o achado 1 estiver corrigido.

Impacto. Acesso contínuo ao conteúdo pago após rejeição ou vencimento, até que a RPC de paywall seja executada.

Controles considerados. A RPC de leitura recusa licença vencida e pode limpar a flag. O React também esconde as aulas. Os dois dependem de serem o caminho usado.

Correção. Na rejeição e no vencimento, gravar is_licensed falso e retirar fixedLessons de data. Não guardar a solução nessa coluna.

Trecho.

```
-- schema.sql:99
FOR SELECT TO authenticated
USING (auth.uid() = user_id OR public.is_admin());

-- adminLicenseService.ts:164
.from('licenses')
.update({ payment_status: 'Rejeitado', valid_until: null })
.eq('id', licenseId);
```

3. [MÉDIA] URL de comprovante e de anexo sem allowlist de host

Categoria. XSS / URLs controladas pelo usuário.

Arquivo:linha. schema.sql:647-649, services/ticketService.ts:88-89, services/adminLicenseService.ts:266-268, pages/AdminPanelPage.tsx:531 e 548.

Fluxo. request_license_order aceita p_receipt_path se casar com um UUID interno ou com qualquer host HTTPS no formato /storage/v1/object/(public|sign)/receipts/. A tabela tickets não tem CHECK em image_url; o INSERT do usuário grava o valor. getReceiptSignedUrl devolve a URL quando ela é HTTPS e contém token=. getAttachmentSignedUrl devolve qualquer URL que comece com http e contenha token=. O painel coloca o resultado em href e, se o pathname terminar em .pdf, em iframe src.

Por que é vulnerável. O esquema javascript: é bloqueado, então não há execução de script na origem do Horium. O host, porém, não é limitado ao projeto Supabase. O administrador que abre o comprovante navega para a página escolhida pelo solicitante.

Quem explora. Usuário autenticado contra o administrador que revisa o pedido ou o chamado.

Pré-condições. Conta autenticada. Para o iframe e a imagem, a CSP de vercel.json restringe frame-src e img-src a *.supabase.co. O link “Abrir em nova aba” não é coberto por frame-src.

Impacto. Phishing do administrador no momento em que ele confere um pagamento. Não é tomada de sessão por XSS.

Controles considerados. getSafeHttpsUrl, regex de storage, sandbox do iframe (sem allow-scripts) e CSP. Eles barram script e esquema perigoso. Não barram a navegação HTTPS.

Correção. Aceitar só caminho relativo na pasta do uid, no bucket privado. Assinar a URL no servidor. Recusar URL absoluta gravada pelo cliente.

Trecho.

```
-- schema.sql:647
p_receipt_path ~*
'^https://[a-zA-Z0-9\.\-]+/storage/v1/object/(public|sign)/receipts/.+'

-- ticketService.ts:88
if (pathOrUrl.startsWith('http') && pathOrUrl.includes('token=')) {
return pathOrUrl;
}
```

4. [BAIXA] Comprovante pode apontar para a pasta de outro usuário

Categoria. IDOR / ownership.

Arquivo:linha. schema.sql:646-653.

Fluxo. p_receipt_path no formato uuid/arquivo.ext é aceito sem comparar o uuid com auth.uid(). O pedido de licença fica associado a esse caminho. Quem faz o upload para o bucket continua preso à própria pasta. Quem abre o arquivo pelo Storage também. O efeito é gravar, no pedido do atacante, a referência do arquivo de outra pessoa.

Por que é vulnerável. A validação confere o formato e não o dono.

Quem explora. Usuário autenticado que já conhece o caminho completo do objeto.

Pré-condições. O nome do arquivo inclui Date.now() e randomUUID() em PlansPage.tsx:100. Sem vazamento desse caminho, a referência não é adivinhável. O atacante não ganha leitura nova: a policy de SELECT do Storage continua valendo.

Impacto. Atribuição indevida de um comprovante alheio a um pedido, se o caminho for conhecido. Severidade baixa por causa dessa pré-condição.

Controles considerados. Policy de upload e de leitura do bucket. Elas impedem ler o arquivo, não impedem citar o caminho.

Correção. Exigir que a primeira pasta seja auth.uid()::text e que o objeto exista nesse prefixo.

Trecho.

```
IF NOT (
p_receipt_path ~* '^[0-9a-f]{8}-.../[a-zA-Z0-9_\.\-]+\.(png|jpe?g|webp|pdf)$'
OR p_receipt_path ~* '^https://.../receipts/.+'
) THEN
RAISE EXCEPTION 'Caminho de comprovante inválido ou inseguro.';
END IF;
```

5. [BAIXA] Chave PIX efetiva quando a variável de ambiente falta

Categoria. Secrets — default inseguro.

Arquivo:linha. pages/PlansPage.tsx:41.

Fluxo. generatePixPayload recebe import.meta.env.VITE_PIX_KEY ou, se o valor é vazio, a string horium.app@gmail.com. Esse endereço também aparece nos termos como e-mail de suporte. O QR e o copia-e-cola são montados no navegador.

Por que é vulnerável. Não é uma chave de API nem um segredo de banco. É um identificador de recebimento que passa a valer sozinho quando o build esquece VITE_PIX_KEY. Não há falha de inicialização.

Quem explora. Não há atacante remoto lendo um segredo privilegiado. O risco é operacional: cobranças do ambiente sem a variável seguem para esse identificador, e o valor fica no bundle.

Pré-condições. Build em que VITE_PIX_KEY não foi definida. Com a variável preenchida, o fallback não entra no payload.

Impacto. Destino de pagamento diferente do configurado, limitado a esse esquecimento. A chave anon histórica não entra neste achado.

Controles considerados. .env.example atual pede um placeholder. Isso não impede o fallback no código.

Correção. Se VITE_PIX_KEY estiver ausente, não gerar cobrança. Falhar o build ou a tela de pagamento.

Trecho.

```
generatePixPayload(
import.meta.env.VITE_PIX_KEY || 'horium.app@gmail.com',
totalPrice
)
```

8. Recomendações

Ordem	Ação	Risco que reduz	Esforço
P1	Isolar fixedLessons sem olhar is_licensed e impedir que schedules.data guarde a solução.	Achado alto. Também corta o fallback do cliente.	Médio
P2	Na rejeição e no vencimento, limpar is_licensed e a solução. Fazer o SELECT não ser o canal da grade paga.	Achado médio de revogação.	Médio
P3	Gravar só caminho relativo do próprio usuário e assinar URL no momento da leitura.	Achados de URL e de pasta do comprovante.	Baixo
P4	Interromper a tela de PIX quando VITE_PIX_KEY não existir.	Fallback de recebimento.	Baixo
P5	Se o projeto phaonqtkhvqkqymqbfjn ainda existir, rotacionar a chave anon publicada no commit a5db3e7.	Higiene. Não é achado contado: o papel do JWT é anon.	Baixo

9. Issues para o GitHub

Cada bloco abaixo está pronto para colar em uma issue. Observações informativas, incluindo a chave anon histórica, não geraram issue.

````
--- ISSUE 1 ---
# [Segurança] Paywall da grade é contornado pela ordem dos triggers

Labels sugeridas: security, alta

## Descrição
Em `schema.sql`, `trg_isolate_schedule_solution` roda antes de
`trg_protect_schedule_license` porque triggers BEFORE disparam em ordem alfabética. O
isolamento decide manter `fixedLessons` enquanto `is_licensed` ainda é verdadeiro. O
trigger seguinte zera a flag para quem não é admin. A solução permanece em
`schedules.data` e o dono a lê pelo SELECT.

## Por que é explorável
Qualquer usuário autenticado pode inserir ou atualizar a própria grade enviando
`is_licensed: true` e `data.fixedLessons`. A chave anon do cliente torna essa chamada
direta. Não depende da tela React, que até mascara o campo.

## Evidência
INSERT/UPDATE do próprio usuário → trigger de isolamento (schema.sql:166) mantém o JSON
→ trigger de licença (schema.sql:108) zera só a flag → policy de SELECT (schema.sql:99)
devolve `data`.

## Arquivo:linha
schema.sql:166
schema.sql:175
services/scheduleService.ts:122

## Trecho de código
```
IF NOT COALESCE(NEW.is_licensed, false) THEN
NEW.data := NEW.data - 'fixedLessons';
END IF;
```

## Impacto
A solução horária paga fica armazenada e legível sem licença aprovada.

## Sugestão de correção
Retirar `fixedLessons` de `schedules.data` para todo não admin, independente de
`is_licensed`. Garantir que o trigger de proteção rode primeiro ou fundir os dois.
Remover o fallback que lê `data.fixedLessons` em `getScheduleSolution`.

## Critérios de aceite
- INSERT e UPDATE com `is_licensed = true` feitos por não admin persistem `is_licensed =
false` e `data` sem a chave `fixedLessons`.
- SELECT da própria grade sem licença aprovada não devolve aulas alocadas.
- `get_schedule_solution` continua recusando grade sem licença vigente.
- Teste de banco cobre a ordem dos triggers.
--- FIM ISSUE 1 ---
````

````
--- ISSUE 2 ---
# [Segurança] Rejeitar ou vencer a licença não remove a solução já gravada

Labels sugeridas: security, média

## Descrição
A policy de SELECT de `schedules` libera a linha inteira ao dono. `rejectLicense` só
altera `licenses`. `is_licensed` e `data.fixedLessons` permanecem. A limpeza existe
apenas dentro de `get_schedule_solution`, se essa RPC for chamada.

## Por que é explorável
O titular autenticado, depois de ter tido `is_licensed` verdadeiro, consulta `schedules`
pela API e lê `data` sem passar pela RPC de paywall.

## Evidência
Aprovação grava `is_licensed = true` → save mantém as aulas no JSON → rejeição em
adminLicenseService.ts:164 não mexe na grade → SELECT em schema.sql:99 devolve `data`.

## Arquivo:linha
schema.sql:99
services/adminLicenseService.ts:164
schema.sql:713

## Trecho de código
```
.from('licenses')
.update({ payment_status: 'Rejeitado', valid_until: null })
.eq('id', licenseId);
```

## Impacto
Conteúdo pago continua disponível ao titular após rejeição ou vencimento, até alguém
invocar a RPC.

## Sugestão de correção
Ao rejeitar e ao detectar vencimento, atualizar a grade: `is_licensed = false` e remover
`fixedLessons` de `data`. Não usar `schedules.data` como cópia da solução.

## Critérios de aceite
- Depois de `rejectLicense`, SELECT do dono não contém `fixedLessons`.
- Licença com `valid_until` anterior a hoje não deixa a solução legível por SELECT,
mesmo sem chamar a RPC.
- A RPC de paywall segue recusando esse caso.
--- FIM ISSUE 2 ---
````

````
--- ISSUE 3 ---
# [Segurança] Comprovante e anexo aceitam URL HTTPS arbitrária

Labels sugeridas: security, média

## Descrição
`request_license_order` aceita um HTTPS qualquer no formato de objeto do Storage.
`tickets.image_url` não tem CHECK. Os serviços devolvem a URL quando ela contém
`token=`, e o painel admin coloca o valor em link e em iframe.

## Por que é explorável
Um usuário autenticado grava a URL no próprio pedido ou chamado. O administrador, ao
conferir o documento, abre o host escolhido pelo usuário. `javascript:` é bloqueado; o
problema é o host.

## Evidência
RPC schema.sql:647 → coluna receipt_url ou image_url → getReceiptSignedUrl /
getAttachmentSignedUrl → AdminPanelPage.tsx:531 (href) e :548 (iframe).

## Arquivo:linha
schema.sql:647
services/ticketService.ts:88
services/adminLicenseService.ts:266
pages/AdminPanelPage.tsx:531

## Trecho de código
```
if (pathOrUrl.startsWith('http') && pathOrUrl.includes('token=')) {
return pathOrUrl;
}
```

## Impacto
Phishing do administrador na revisão do pagamento ou do chamado. A CSP limita imagem e
iframe a *.supabase.co; o link em nova aba não.

## Sugestão de correção
Persistir somente caminho relativo. Na leitura, gerar signed URL do bucket. Recusar URL
absoluta.

## Critérios de aceite
- RPC e INSERT de ticket recusam `https://` externo e `javascript:`.
- O visualizador só recebe signed URL do bucket do projeto.
- Teste com host externo não abre href nem iframe.
--- FIM ISSUE 3 ---
````

````
--- ISSUE 4 ---
# [Segurança] Caminho do comprovante não é amarrado ao usuário autenticado

Labels sugeridas: security, baixa

## Descrição
A regex de `p_receipt_path` aceita qualquer UUID como primeira pasta. Não compara essa
pasta com `auth.uid()`.

## Por que é explorável
Se o caminho de um objeto alheio for conhecido, o atacante referencia esse objeto no
próprio pedido. A leitura do arquivo continua limitada pela policy do Storage. O nome
gerado em PlansPage usa timestamp e UUID, então não é adivinhável.

## Evidência
request_license_order schema.sql:646-653 valida formato e grava `receipt_url` sem checar
o prefixo.

## Arquivo:linha
schema.sql:647

## Trecho de código
```
p_receipt_path ~*
'^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}/[a-zA-Z0-9_\.\-]+\.(png|j
pe?g|webp|pdf)$'
```

## Impacto
Associação do comprovante de outra pessoa a um pedido, condicionada ao conhecimento do
caminho. Sem leitura nova do arquivo.

## Sugestão de correção
Exigir `(storage.foldername(p_receipt_path))[1] = auth.uid()::text` e a existência do
objeto nessa pasta.

## Critérios de aceite
- Caminho com UUID diferente de `auth.uid()` é recusado.
- Caminho na pasta do próprio usuário continua aceito.
- URL absoluta continua recusada pelo critério da issue 3.
--- FIM ISSUE 4 ---
````

````
--- ISSUE 5 ---
# [Segurança] PIX de cobrança usa identificador fixo se a variável não existir

Labels sugeridas: security, baixa

## Descrição
`PlansPage` gera o payload PIX com `VITE_PIX_KEY` ou, em ausência,
`horium.app@gmail.com`.

## Por que é explorável
Não é roubo de credencial. Em um build sem a variável, a cobrança exibida ao usuário
aponta para esse identificador, que também está no código-fonte.

## Evidência
pages/PlansPage.tsx:41 passa o fallback para `generatePixPayload`.

## Arquivo:linha
pages/PlansPage.tsx:41

## Trecho de código
```
generatePixPayload(import.meta.env.VITE_PIX_KEY || 'horium.app@gmail.com', totalPrice)
```

## Impacto
Pagamentos do ambiente mal configurado seguem para o identificador embutido. Sem a
variável ausente, o fallback não é usado.

## Sugestão de correção
Não montar QR nem copia-e-cola quando `VITE_PIX_KEY` estiver vazio. Tratar isso como
erro de configuração.

## Critérios de aceite
- Build sem `VITE_PIX_KEY` não gera payload PIX.
- Build com a variável usa somente esse valor.
- O identificador fixo deixa de aparecer no bundle.
--- FIM ISSUE 5 ---
````

Fim do relatório. Cinco achados confirmados, cinco issues acionáveis, nenhuma categoria deixada sem análise.